Presented to the College of Computer Studies

De La Salle University - Manila

Term 1, A.Y. 2025-2026

In partial fulfillment of the course

In CSINTSY N25

Major Course Output #1

Group: INTSYCURITIES

Submitted by:

Enciso, Gabriel Jeremy C.

Ortha, Gian Lorenzo C.

Sy, Wesley Brian T.

Submitted to:

Mr. Thomas Tiam Lee

June 26, 2026

I. Sokobot Algorithm

Our Sokobot uses a Greedy Best First Search (GBFS) algorithm. It holds the states in three separate structures: the explored and frontier, which are standard for the GBFS algorithm, and another set to store the pruned states. Both the explored and pruned sets are implemented as hash sets for fast lookup, and the frontier is implemented as a priority queue, so that the state with the lowest heuristic value is processed first. The pruned set stores states that were identified as dead states, such as those that are deadlocked, so that we can avoid adding these to the frontier altogether.

Even before the search begins, the solver will pre-calculate a set of deadlocks on the map for future use. An initial GameState object is put in the frontier that contains the starting position of the player and the positions of the boxes. The first state is explored, and from there, the solver determines what the valid actions from that state are. It then checks each new candidate state before adding it to the frontier. It first checks whether the state has been pruned, and if so discards it. It then checks if the state contains a deadlock using the previously calculated deadlock sets, and if it does, add it to the pruned set and then discard it. Next it checks whether it has been explored and if so, discards it. Once it passes all these checks, this candidate state can be added to the frontier.

The state's heuristic is calculated and stored before it is put into the frontier. The heuristic is calculated as the sum of all the minimum Manhattan distances of each box that is not in a goal from the closest goal in relation to the box. The solver then moves to the next iteration, selecting the state with the lowest heuristic and exploring it, until the solved state is found. Each GameState keeps track of its predecessor, therefore, to construct the solution, the actions leading to each state are traced backwards and then reversed, giving a sequence of moves to solve the map.

II. Evaluation and Performance

Since the main objective is to solve the provided maps within a 15-second time limit, we decided to have the solver focus on finding a solution as quickly as possible rather than finding the shortest or most optimal path. Therefore, our solver might not produce the solution with the lowest number of moves, and may even produce solutions with a lot of unnecessary backtracking (in terms of player movement).

We tested its performance on a laptop with an Intel Core i5 processor, 16GB of RAM, and no discrete graphics card. The bot should run well on a normal laptop even without a dedicated graphics card, because it was not necessarily programmed to utilize it. Based on our testing, our bot solved most of the provided maps within 1 second. The only unsolved maps are original2 and original3. Testing other maps that we found online,

it seems that the bot's performance falls off significantly once the map has more than 7-8 boxes.

Some strengths of the bot include its efficient iteration by using hash sets for almost all data that need some form of lookup, and its state pruning, that prevents the exploration of dead states. The bot also has relatively low memory usage, as we employed various strategies to generally avoid creating a lot of objects to save on unnecessary memory overhead. One of its weaknesses, however, is its heuristic. Due to the naive nature of heuristics, the bot will generally start to struggle with more complex maps with more boxes, as there are a lot more potential moves that a simple heuristic-based system cannot take into consideration (i.e. moving one box out of the way to let another one pass).

III. Challenges

Starting the project, one of the early challenges we had was talking about what the state of the game is, this was around weeks 5-6 when we've only started getting the hang of determining what states are. For our bot, we decided to hold the location of the boxes and the player as part of our states. After agreeing how to represent our states, we had to figure out which search algorithm to use. Although we did end up using Greedy Best First Search (GBFS), we had a discussion whether to use A* as our search algorithm. With the project having a limit of 15-second thinking time GBFS provided us with a more practical approach, because it prioritizes completion of the sokoban whereas A* prioritizes finding the shortest solution possible, which might result in unnecessary overhead for the search algorithm.

However, picking GBFS introduced us to another problem, because its performance relies heavily on heuristics. The early versions of the bot run but each state holds too much map data, and the bot struggles with larger maps like the four-boxes or five-boxes. Some of the optimizations were made to help solve more levels faster such as, HashSets for faster lookup this was used to check whether a state was already explored or already pruned. Fixing repeated state exploration, where the bot skips a state if it has already been explored, pruned, or identified as a deadlock. Improving heuristics, where instead of using the average distance of boxes to goals, Gian opted to go for a heuristic using each box that is not yet on a goal to its closest available goal. Adding deadlock detection, since boxes cannot be pulled, while checking candidate states, the bot checks if a box is in a dead position. Adding that state to the pruned set, and to be ignored instead of being searched further. Lastly what AI suggested was to flatten 2D positions into 1D values, `flattenedPos = (y * mapWidth) + x.` This makes box positions easier to store in sets, compare between states, and hash for faster lookup.

Since GBFS creates and checks as many as possible, stripping away expensive operations that slowed the bot down was another challenge. Initially represented the possible actions from one state as a string i.e. “lrd.” Switched instead to using a number, setting its 4 least significant bits abcd to 1 or 0 for a valid or invalid left, right, up, or down movement respectively. Having the map data pass often passed into each game state, which was expensive, the mapData was made to only be passed through methods instead only when needed.

IV. Table of Contributions

Gabriel Enciso
❖ Conceptualization of GameState, and Search Algorithm ❖ Writing contents of the report (challenges)
Gian Ortha
❖ Conceptualization and implementation of the GameState object ❖ Conceptualization and implementation of the logic for the bot (GBFS algo, PriorityQueue’s HeuristicsComparator clas) ❖ Conceptualization, implementation, and optimization of the heuristic calculation method. ❖ Conceptualization and implementation of deadlock detection and dead state pruning ❖ Optimization of code (for the bot to run faster and also use less memory, i.e. using HashSets for faster lookups, and prioritizing the use of primitive types over reference types when possible) ❖ Creation of the benchmark script for easier testing of the bot ❖ Writing of the outline of the contents of the report ❖ Proofreading of the report
Wesley Sy
❖ Performed map testing ❖ Created report outline ❖ Did SokoBot Algorithm and the Evaluation and Performance section of the report